



PDF Download
3678185.pdf
19 January 2026
Total Citations: 5
Total Downloads:
1213

Latest updates: <https://dl.acm.org/doi/10.1145/3678185>

RESEARCH-ARTICLE

Utilizing classical programming principles in the Intel Quantum SDK: implementation of quantum lattice Boltzmann method

Published: 14 January 2025
Online AM: 17 July 2024
Accepted: 21 June 2024
Revised: 04 June 2024
Received: 29 September 2023

[Citation in BibTeX format](#)

TEJAS SHINDE, University of Jyväskylä, Jyväskylä, Central Finland, Finland

LJUBOMIR BUDINSKI, University of Novi Sad, Novi Sad, Vojvodina, Serbia

OSSI NIEMIMÄKI

VALTTERI LAHTINEN

HELENA LIEBELT, Deggendorf Institute of Technology, Deggendorf, Bayern, Germany

RUI LI, Deggendorf Institute of Technology, Deggendorf, Bayern, Germany

Open Access Support provided by:

Deggendorf Institute of Technology

University of Novi Sad

University of Jyväskylä

Utilizing classical programming principles in the Intel Quantum SDK: implementation of quantum lattice Boltzmann method

TEJAS SHINDE, Quanscient Oy, Tampere, Finland and Applied Computer Science, University of Jyväskylä Faculty of Information Technology, Jyväskylä, Finland

LJUBOMIR BUDINSKI, Quanscient Oy, Tampere, Finland and University of Novi Sad Faculty of Technical Sciences, Novi Sad, Serbia

OSSI NIEMIMÄKI, Quanscient Oy, Tampere, Finland

VALTTERI LAHTINEN, Quanscient Oy, Tampere, Finland

HELENA LIEBELT, Applied Computer Science, Deggendorf Institute of Technology, Deggendorf, Germany

RUI LI, Deggendorf Institute of Technology, Deggendorf, Germany

We explore the use of classical programming techniques in implementing the quantum lattice Boltzmann method in the Intel Quantum SDK—a software tool for quantum circuit creation and execution on Intel quantum hardware. As hardware access is limited, we use the state vector simulator provided by the SDK. The novelty of this work lies in leveraging classical techniques for the implementation of quantum algorithms. We emphasize the refinement of algorithm implementation and devise strategies to enhance quantum circuits for better control over problem variables. To this end, we adopt classical principles such as modularization, which allows for systematic and controlled execution of complex algorithms. Furthermore, we discuss how the same implementation could be expanded from state vector simulations to execution on quantum hardware with minor adjustments in these configurations.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Applied computing** → *Physical sciences and engineering*;

Additional Key Words and Phrases: Quantum computing, quantum algorithm, Intel Quantum SDK

This research was partially supported by the Business Finland project 9820/31/2022 Quantum-Native Multiphysics.

Authors' Contact Information: Tejas Shinde, Quanscient Oy, Tampere, Pirkanmaa, Finland and Applied Computer Science, University of Jyväskylä Faculty of Information Technology, Jyväskylä, Finland; e-mail: tejas.m.shinde@student.jyu.fi; Ljubomir Budinski, Quanscient Oy, Tampere, Pirkanmaa, Finland and University of Novi Sad Faculty of Technical Sciences, Novi Sad, Serbia; e-mail: ljubomir.budinski@quanscient.com; Ossi Niemimäki, Quanscient Oy, Tampere, Pirkanmaa, Finland; e-mail: ossi.niemimaki@quanscient.com; Valtteri Lahtinen, Quanscient Oy, Tampere, Pirkanmaa, Finland; e-mail: valtteri.lahtinen@quanscient.com; Helena Liebelt, Applied Computer Science, Deggendorf Institute of Technology, Deggendorf, Bavaria, Germany; e-mail: helena.liebelt@th-deg.de; Rui Li, Deggendorf Institute of Technology, Deggendorf, Germany; e-mail: rui.li@th-deg.de.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s).

ACM 2643-6817/2025/01-ART9

<https://doi.org/10.1145/3678185>

ACM Reference Format:

Tejas Shinde, Ljubomir Budinski, Ossi Niemimäki, Valtteri Lahtinen, Helena Liebelt, and Rui Li. 2025. Utilizing Classical Programming Principles in the Intel Quantum SDK: Implementation of Quantum Lattice Boltzmann Method. *ACM Trans. Quantum Comput.* 6, 1, Article 9 (January 2025), 18 pages. <https://doi.org/10.1145/3678185>

1 Introduction

In the field of engineering, industrial and scientific simulations place substantial demands on computational resources. Advances in computing hardware, such as high-performance computing clusters and GPUs, are catering to increasing computational demands, but the ability to shrink transistors is hitting physical limits, threatening the well-known Moore's law [Waldrop 2016]. Quantum computing has received substantial attention over the past few years as a means of pushing the envelope. New algorithms have been developed, showing algorithmic efficiency over classical computing. For example, there is a potential quantum speed-up in applications such as linear algebra [Lloyd et al. 2020; Qian et al. 2019; Ambainis 2010], quantum chemistry [Cao et al. 2019], optimization [Kerenidis and Prakash 2020; Farhi et al. 2014] and machine learning [Garg and Ramakrishnan 2020; Sharma 2020].

With the development of algorithms, there is a need for refined methods and techniques for smooth and robust implementation. Implementing the quantum circuits in a reusable and scalable manner, and integrating them into possible applications poses a new challenge. At its core, this challenge arises from the intricate nature of quantum algorithms, which usually rely on complex sequences of quantum gates acting on qubits. Achieving reusability is challenging due to the constraints and the specificity of each application. Scalability is vital for building larger optimized quantum algorithms and ensuring efficient utilization of the resources as systems grow in size.

The main contribution of this work is the use of classical programming principles in implementing quantum algorithms in the Intel Quantum SDK [Khalate et al. 2022]. The Intel Quantum SDK is a C++ based quantum programming framework tailored for hybrid quantum-classical programming. For this work, we have chosen the **quantum lattice Boltzmann method (QLBM)** as an example for the implementation. The QLBM is a promising quantum algorithm for computational fluid dynamics simulations. It provides a quantum-native way to solve transport phenomena, with recent advancements showing its potential for quantum efficiency [Budinski 2021, 2022; Budinski et al. 2023]. In this article, we focus on the QLBM algorithm for the advection-diffusion equation as presented in Budinski [2021]. Additionally, we provide a brief explanation on expanding the implementation to 2D advection-diffusion [Budinski 2021, Section 3.2] and Navier–Stokes equations using the stream function-vorticity formulation [Budinski 2022]. However, we do not aim at expanding or improving the QLBM algorithm itself in this work but rely solely on the already-existing knowledge.

Through this example, we explore the programming design principles in the context of quantum algorithms. We focus on the modular approach in forming generic building blocks for the complex algorithm, such that can be easily scaled and reused in different configurations. We also outline possibilities for hybrid design, for which the Intel Quantum SDK is particularly suitable.

This article is structured as follows. Section 2 provides a concise overview of the Intel Quantum SDK. In Section 3, we briefly recap the **lattice Boltzmann method (LBM)** and discuss the quantum circuit implementation in general. In Section 4, we begin with an analysis of how classical techniques are employed for the implementation of quantum algorithms in the Intel Quantum SDK. The section concludes with an exploration of expanding the implementation to address 2D

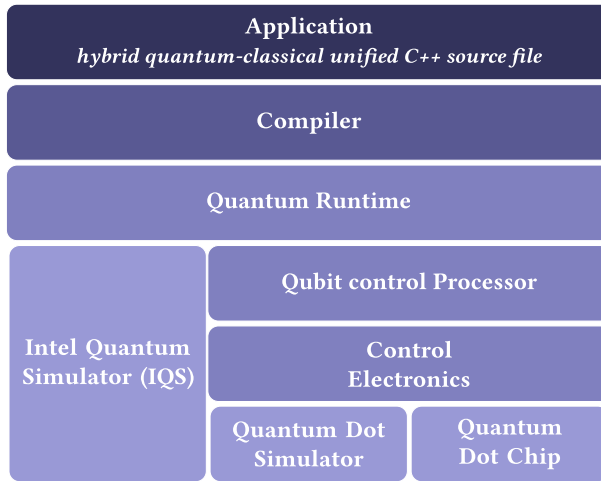


Fig. 1. Overview of Intel Quantum SDK as described in Khalate et al. [2022].

advection-diffusion and Navier–Stokes equations, accompanied by numerical validation using a simple example.

2 Intel Quantum SDK

The Intel Quantum SDK is an LLVM¹ based environment that allows the writing of hybrid quantum-classical algorithms on computational systems, targeting variational quantum algorithms in particular [Khalate et al. 2022]. The SDK follows a similar approach as a classical hardware accelerator environment, which allows one to write instructions targeted for specialized hardware such as GPU and FPGA in order to increase efficiency. The SDK aims at making it efficient to run hybrid quantum-classical parts of computational problems, where the quantum hardware acts as an accelerator. The Intel Quantum SDK enables quantum kernels for quantum instructions targeted for the quantum hardware, and classical methods for the classical instructions targeted towards the CPU within the same algorithm. Separating the instructions benefits the use of hybrid quantum and classical computing.

The SDK provides a C++ based programming interface based on the circuit model for quantum computation. The SDK is developed with the Intel quantum hardware in mind, but these devices are not yet available to the general public. Instead, the SDK provides a full-state simulator class with API calls to set up a CPU-based simulator and provides access the quantum state during the simulation. It allows users to simulate and test their program in an ideal case without access to actual quantum hardware. The quantum state is accessed through methods that return the conditional probabilities of the qubits and complex amplitudes of the state space. Figure 1 gives an overview of the Intel quantum computing stack.

The version 1.0 of the Intel Quantum SDK added a Python interface to the existing framework. This helps convert OpenQASM2 [Cross et al. 2017] circuit instructions to Intel Quantum SDK compatible C++ source code. The Python interface also allows to compile and run the quantum circuits directly. We make use of this in the implementation to add circuit functionalities that would have been otherwise difficult to realize through the C++ interface at the time of writing.

¹A collection of compiler and toolchain technologies.

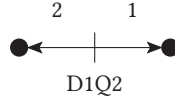


Fig. 2. 1D lattice arrangement.

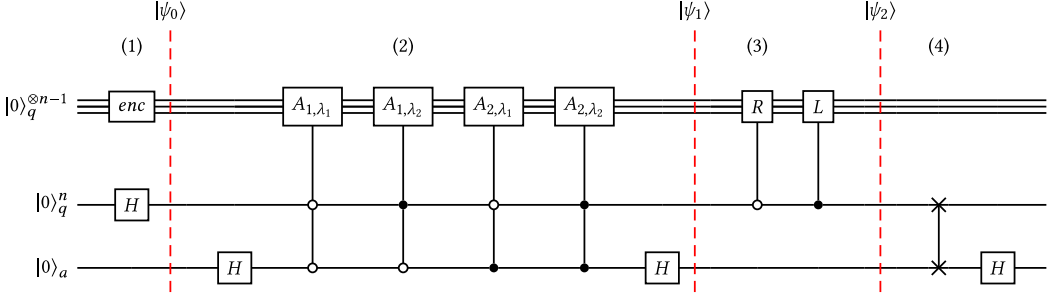


Fig. 3. Quantum circuit to solve the 1D advection-diffusion equation with the lattice Boltzmann method.

3 QLBM

The LBM is a computational fluid dynamics technique for simulating complex fluid flows. Traditional methods in computational fluid dynamics, such as finite difference, finite volume, and finite element methods, rely on solving the Navier–Stokes equations to model fluid flow. These techniques involve complex discretization schemes and extensive computational resources, often leading to challenges in handling complex boundary conditions and multiphase flows. In contrast, the LBM offers a more intuitive and flexible approach by simulating fluid dynamics through the evolution of particle distribution functions on a lattice grid—see for example [Mohamad 2019] for a comparison of the LBM with the traditional finite difference method in different scenarios.

In particular, we are interested in the LBM as a promising method for solving fluid dynamics problems with quantum computers and use the simple advection-diffusion model [Budinski 2021] as an example of a hybrid computation scheme. For more recent advancements on quantum implementations see for example [Schalkers and Möller 2024] and [Schalkers and Möller 2024].

The LBM operates on a lattice grid where each node represents a fluid particle distribution function. Through a series of collision and propagation steps, these functions evolve over time to simulate fluid behavior.

The LBM follows a mesoscopic approach and bridges the gap between molecular dynamics and macroscopic fluid dynamics, offering insights into various flow phenomena at different scales. [Krueger et al. 2016]

A D1Q2 lattice arrangement is shown in Figure 2, where DnQm stands for n dimensions and m velocity vectors.

The D1Q2 arrangement consists of two velocity vectors $e_1 = 1, e_2 = -1$ for the distributions f_1 and f_2 , respectively. At an instance in time, two particles exist at a site, moving right and left in the streaming process. The weighting factors for the D1Q2 are $1/2$ for both distributions f_i .

As a simple example, we study the QLBM solution to the advection-diffusion equation. The quantum algorithm for the D1Q2 QLBM can be divided into four major steps: encoding, collision, propagation, and the calculation of macroscopic variables. Each of these is illustrated in the circuit implementation in Figure 3. The steps proceed as follows:

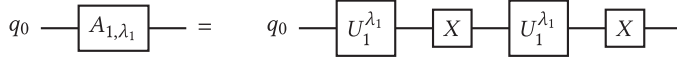


Fig. 4. Quantum circuit to implement the A_{1,λ_1} operator.

- (1) Encoding: In the encoding step, we set up two quantum registers q and a . To encode the vector into quantum states and do the computation we require the quantum working register q . The working register encodes the vector with concentration $C(x, t)$ having qubits equal to $\log_2(2M)$, where M is the lattice size. The ancilla register a consists of only one qubit and is used in the controlled operation in the collision step, as shown in Figure 3. To achieve the initial state, external state preparation is used, for example, the state preparation procedure proposed by Shende et al. [2006]
- (2) Collision: The collision step is a point-wise multiplication between the vector C and the block diagonal matrix according to

$$f_i^{eq}(\vec{x}, t) = \omega_i C(x, t) \left(1 + \frac{e_i \cdot \vec{u}}{c_s^2} \right), \quad (1)$$

excluding the depended variable vector $C(x, t)$. Since the block diagonal matrix is non-unitary, a linear combination of unitaries approach is used, providing two unitary diagonal operators A_1 and A_2 . The quantum circuit for the collision part is given in Figure 3 in between the states $|\psi_0\rangle$ and $|\psi_1\rangle$. The decomposition of the A_1 and A_2 gates in the circuit Figure 3 is given in Figure 4 involving a phase gate and an X gate.

- (3) Propagation: In the propagation step, we use the quantum walk [Childs 2009; Low and Chuang 2019] procedure for the streaming of the distributions. In this step, the distribution function f_1 moves to the right with speed $e_1 = 1$ and the distribution function f_2 moves to the left with speed $e_2 = -1$. This is achieved by the operators illustrated in Figure 3, R and L for right shift and left shift, respectively.
- (4) Macroscopic variables: The last step involves the calculation of the macroscopic variables, density or velocity, this is possible through the point-wise addition of the two quantum states. As the equation [Budinski 2021, Equation 11] indicates that both of the distribution functions are located in the subsystem controlled by the ancilla qubit $|0\rangle_a$, for the point-wise addition at first a swap gate is added between the last qubit q_n of the working register q and the ancilla qubit a . This operation switches the states of the working register controlled by the $|0\rangle_a$ and $|1\rangle_a$ in the ancilla register, resulting in the positioning of the sub-state of the second distribution function, controlled by $|0\rangle_a$, to the state controlled by $|1\rangle_a$. The implementation of this is simple as given in Figure 3, just the swap and a Hadamard gate on the ancilla at the end.

The LBM is a time-marching algorithm, and the above outline covers only one-time step. It is not detailed in Budinski [2021] how to connect time steps purely inside a quantum circuit, and we will not attempt to present such a solution here. We use the re-encoding of the initial state at each time step as an example of a hybrid design and study the implementation from this particular angle. Note that following this approach likely destroys the possible speed-up advantages of the quantum algorithm. However, it provides an interesting point of study from the program design perspective, which can be useful even after the time-stepping problem for the QLBM has been resolved. Recent attempts towards efficient time-step concatenation can be found, for example, in Schalkers and Möller [2024].

The gate complexity of the QLBM depends on the resolution of the computational domain, as detailed in Budinski [2021, Section 3.3]. It is highlighted that the encoding step is the most

resource-intensive, requiring a significant number of state preparation steps and CX gates, which depends on the problem to be solved—in the most general case, this cost is exponential. In the collision step, the complexity is not directly affected by the resolution of the computational domain: increasing the number of qubits to achieve a higher resolution does not change the size of the ancillary register, and the number of multi-qubit gate operations required in the q register is constant. The propagation step is influenced by the resolution of the computational domain, and the depth of this subroutine increases logarithmically. The complexity of the propagation step can be optimized for example by using the parallel state shift method [Budinski et al. 2023]. The overall complexity of the algorithm, excluding the encoding step, is determined to be $O(\log_2(\alpha D))$, where α represents the number of velocity vectors and D depends on the domain resolution ($D = 2M$ for $D1Q2$).

4 Quantum Algorithm Implementation in Intel Quantum SDK

In the form presented above, the quantum lattice Boltzmann algorithm is a hybrid method, although not a variational one. This hybrid property makes it, nevertheless, an interesting example for implementation on platforms that support hybrid classical and quantum computing, such as the Intel Quantum SDK.

In this section, we detail such an implementation and explore the use of modularization with the aim of creating easily reusable and scalable quantum circuits.

4.1 Implementation

Due to the complex nature of the quantum lattice Boltzmann algorithm, it becomes essential to employ specific quantum software engineering techniques for the implementation. A general idea is proposed in Khan et al. [2022, Section 3] about the iterative development cycle and infrastructure for quantum program development: to construct adaptable quantum circuits optimized for execution. In this section, we focus on cycle 2, especially the implementation, the quantum algorithm design cycle, where we adhere to an iterative process for creating and testing various configurations. A modular design methodology is followed for the implementation in the Intel Quantum SDK, similar to the one described in Zhao [2021].

The quantum circuit for the algorithm is provided in Figure 3. Figure 6 illustrates how the hybrid time-stepping is performed in the algorithm. The time-step process explained here is specifically for the purpose of implementing the algorithm in the Intel Quantum SDK, and we have chosen it as an example of the hybrid approach in the algorithm design. To implement the time-stepping, the algorithm is divided into three modules, following the modular design methodology [Zhao 2021, Section 5.4], as described below.

- (1) Pre-processing: preparing the initial encoding circuit.
- (2) Solver: solving the four steps discussed in Section 3 in the Intel Quantum SDK.
- (3) Post-processing: extracting the state vector from the Intel Quantum SDK and post-processing the data.

4.1.1 The Top-down Approach. In this study, we employ the top-down approach to software engineering, adhering to the Single Responsibility Principle. This methodology begins with a high-level design of the overall system, ensuring that each module has a single responsibility. The system is progressively decomposed into smaller, more detailed components, with each module focusing on a specific task. This decomposition promotes modularization, facilitating easier maintenance and scalability.

The Intel Quantum SDK provides a set of quantum logic gates that can be used to create quantum circuits, with each circuit forming a quantum kernel. The goal is to use these quantum kernels and create mutable quantum circuits (adaptable to different configurations) for the different steps

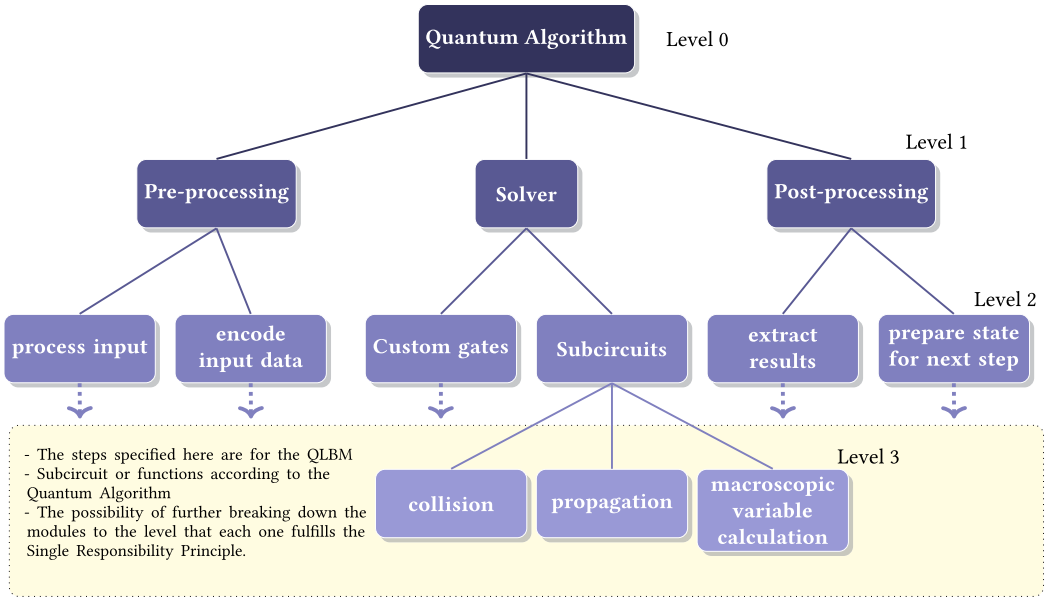


Fig. 5. Illustration of progressive decomposition of the quantum algorithm into smaller implementable modules—specifically for QLBM decomposing into modules such as Custom gates and Subcircuits shown.

involved in the QLBM solver. The hardware accelerator design helps us in the modularization of the quantum circuit in Figure 3 and to write a quantum kernel for each module, which we will call *subcircuits*. The different subcircuits are according to the steps involved such as the encoding, collision, propagation, and the calculation of macroscopic variables.

The subcircuits might include gates controlled by multiple qubits. All but one of the quantum logic gates available in the Intel Quantum SDK are one- and two-qubit gates, the exception being the three-qubit Toffoli gate. For the implementation of multi-qubit controlled gates, the decomposition method described in the book [Nielsen and Chuang 2010, Section 4.3] is utilized. We call such decomposed gates *custom gates*. With the help of the basic logic gates and the custom gates, we have the basic building blocks for the subcircuits, minimizing the size of the modules.

The flowchart in Figure 5 outlines the structure of a quantum algorithm, divided into three main stages: Pre-processing, Solver, and Post-processing. In the pre-processing stage, input data is processed and encoded. The solver stage includes custom gates and subcircuits, with subcircuits further broken down into collision, propagation, and macroscopic variable calculation. The post-processing stage involves saving results and preparing the state for the next step. Each level of the flowchart provides a clear hierarchical view, from high-level algorithm structure down to specific computational tasks.

The quantum algorithm can be modularized according to the flowchart, allowing for clear separation of tasks and easier management. This ensures each stage is independently developed and tested, facilitating parallel development. The hierarchical structure promotes scalability and adaptability, making it suitable for various quantum algorithms by customizing each module to fit specific requirements.

4.1.2 Pre-processing. The QLBM requires that initial values for the advection-diffusion problem be provided in the form of an amplitude vector. These initial values can be arbitrary and may vary according to the specific problem. The Intel Quantum SDK does not include a built-in

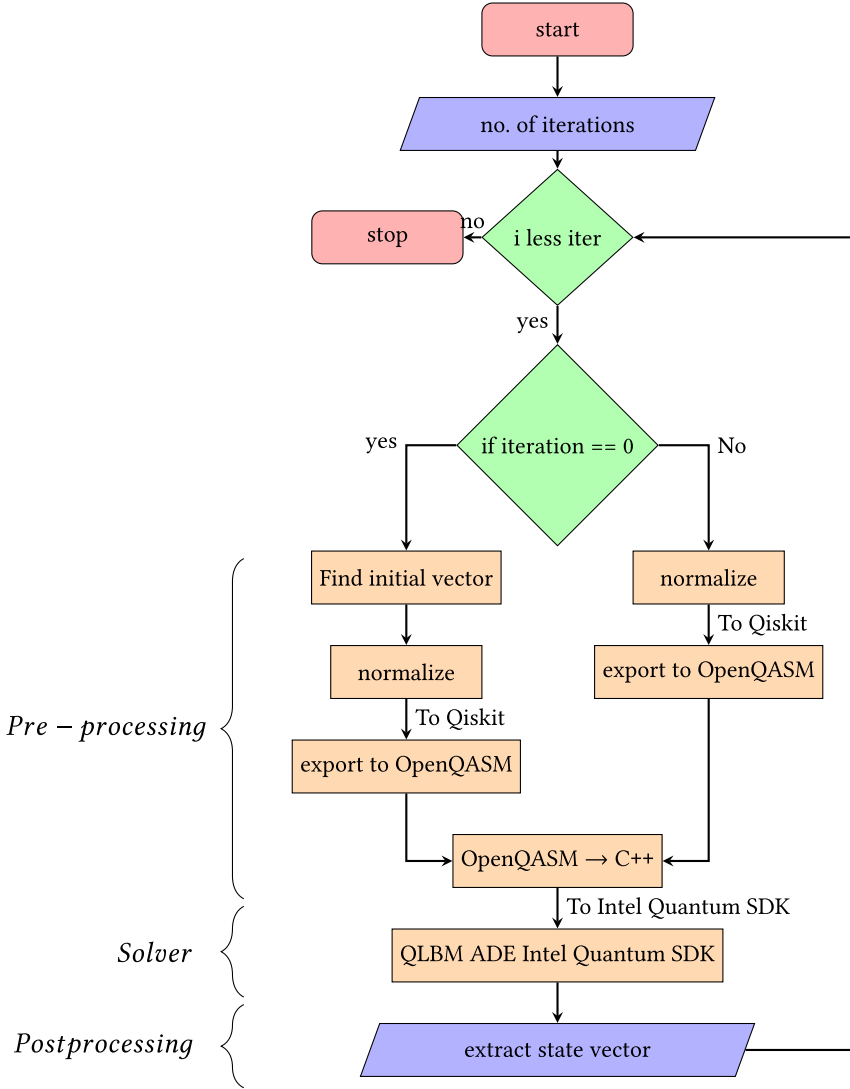


Fig. 6. Overview on the hybrid time-stepping.

amplitude encoding algorithm, and this makes it necessary to either build one with the SDK or use some external amplitude encoding routine. There are several state preparation algorithms documented in the literature, such as the ones referenced in Shende et al. [2006], Zhang et al. [2022], and Zylberman and Debbasch [2023]. To study the hybrid time-stepping implementation on the full-state simulator, we use the Qiskit [Qiskit contributors 2023] state preparation class as an external routine, which is built upon the reverse iterative procedure proposed by Shende et al. [2006, Section 4].

We implement this step using the Python interface which allows the conversion from OpenQASM to C++ source code compatible with the Intel Quantum SDK. The flowchart in Figure 6 illustrates the steps involved in encoding the initial state. Note that any other amplitude encoding routine could just as well be used in the place of the Qiskit algorithm; we have simply chosen it out of convenience.

ALGORITHM 1: Encoding**Require:** initial state = $[\]$, num-qubits, lattice-size**Ensure:** Qasm file with Quantum Circuit

- 1: normalize initial state
- 2: initialize quantum Circuit (qc)
- 3: (qc) use qiskit initialize function with specified num-qubits and initial state
- 4: resulting qc passed to Qiskit transpile
- 5: transpiled circuit converted and saved as OpenQASM file

4.1.3 Solver. This module gathers together the implementation of four algorithmic steps: encoding, collision, propagation, and the calculation of macroscopic variables. Section 4.1.2 outlined the implementation of the encoding step. We then have three steps to implement inside the SDK, and we divide each step into subcircuits with different quantum kernels, allowing for flexibility with the circuit parameters. Additionally, multi-qubit custom gates are generated for these subcircuits. The generation of these subcircuits culminates in the main function which calls all the defined subcircuits, as outlined in Algorithm 2. The subcircuits are formed using the custom gates and the definition of custom gates is given below.

ALGORITHM 2: Solver: Main function**Require:** subcircuits**Ensure:** probabilities

- 1: initialize the backend
- 2: quantum kernel encoding ▷ Calling the subcircuits
- 3: quantum kernel collision
- 4: quantum kernel propagation
- 5: quantum kernel macros
- 6: get probabilities and amplitude and save the data

Initialize backend. In the main function, the initial step involves setting up a backend to act as our quantum hardware. The Intel Quantum SDK provides a full-state simulator, which serves as our window to testing the algorithm in the ideal setting.

Once we have access to actual quantum hardware, we can set up and prepare the backend to work with that specific type of quantum hardware. This is really the only significant change we will need to make. The core structure, algorithms, and methods we are using and discussing can be consistently reused regardless of the backend. Essentially, adjusting to real quantum hardware mainly involves configuring the backend while keeping everything else intact.

Custom gates. The Intel Quantum SDK provides a set of basic logic gates, with the help of these logic gates the different multi-qubit gates can be derived. We use these custom gates as building blocks for the subcircuits.

In the quantum circuit given in Figure 3, for the collision subcircuit, there are two-qubit phase and two-qubit X gates. The two-qubit X gate or Toffoli is one of the logic gates in the set of given basic logic gates. The two-qubit controlled phase gate can be derived by using one- and two-qubit gates from the gate set. The decompositions illustrated for a two-qubit phase gate in Figure 7 and for a five-qubit controlled X gate in Figure 8 are based on the general multi-qubit controlled gate decomposition explained in Nielsen and Chuang [2010, Section 4.3]. The pseudo-code for the two-qubit-controlled phase gate decomposition is given in Algorithm 3. With this formulation, we have created a basic building block custom gate named CCPHASE.

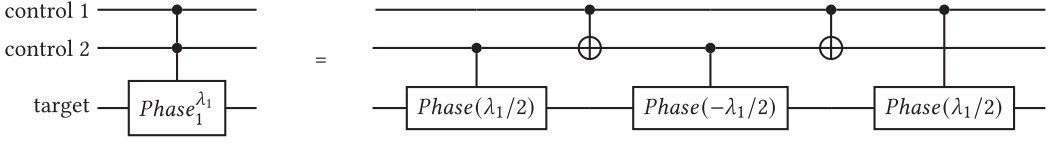


Fig. 7. Decomposition of two-qubit controlled phase gate: CCPHASE.

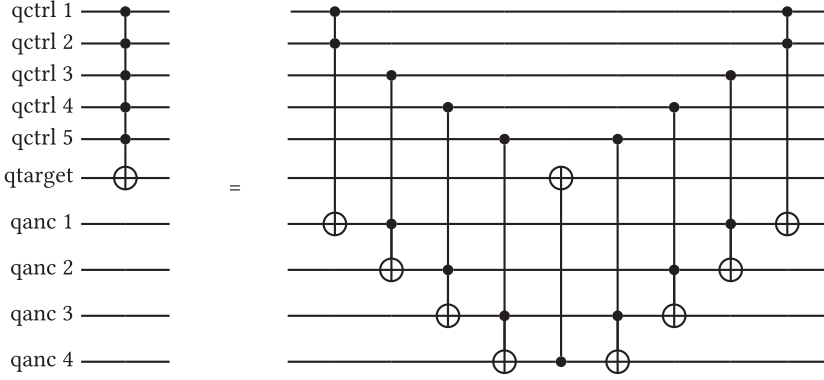


Fig. 8. Decomposition of five-qubit controlled X gate (MCX5) using ancilla qubits.

ALGORITHM 3: Custom gate: CCPHASE – two-qubit controlled phase gate**Require:** const int (control qubit1, control qubit2, target qubit, phase angle)**Ensure:** basic gate

- 1: **function** (QUANTUM KERNEL) CCPHASE(control 1, control 2, target qubit, angle)
- 2: CPhase(qubit control 2, target, angle / 2);
- 3: CX(qubit control 1, qubit control 2);
- 4: CPhase(qubit control 2, target, - angle / 2);
- 5: CX(qubit control 1, qubit control 2);
- 6: CPhase(qubit control 1, target, angle / 2);
- 7: **end function**

Similarly, for the multi-qubit gates, there is the additional requirement of ancilla qubits as shown in the five-qubit example in Figure 8. The number of ancilla qubits required is one less than the number of control qubits. The pseudo-code for this particular five-qubit decomposition is given in Algorithm 4.

Subcircuits. In the Section 4.1.3, we explained the basic building blocks needed for constructing the subcircuits. This section elucidates the formation of different subcircuits mentioned in Section 3 using custom gates and the basic gates provided by the SDK. The process of constructing the subcircuits is identical to that of custom gates, the only difference being that the subcircuits are one level higher and make use of the custom gates. It is important to note that the qubit parameters for the gates must be unambiguous, allowing the compiler to differentiate between quantum and classical instructions. Currently, this is a disadvantage when creating generalized quantum custom gates, as it requires precise qubit specification during the design.

The collision subcircuit requires the two-qubit controlled phase gate and a Toffoli with Hadamard gates on either side. The CCPHASE custom gate is used to implement this subcircuit. The subcircuit also contains some controls with a value of zero instead of one. To convert the

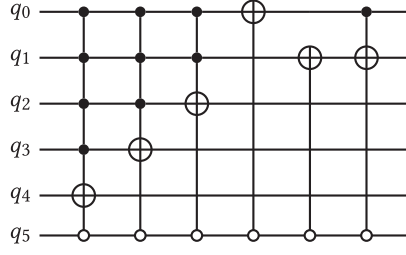


Fig. 9. The right shift operator for a circuit of five working qubits. The last qubit controls on the direction of the shift.

ALGORITHM 4: Custom gate: five-qubit controlled X gate

Require: const int (control 1 - 5, target, phase angle)

Ensure: basic gate

```

1: function (QUANTUM KERNEL) MCX5(control 1 - 5, target qubit, ancilla 1 - 4)      ▷ 5 qubit
   controlled X gate
2:   Toffoli(control 1, control 2, ancilla 1)
3:   Toffoli(control 3, ancilla 1, ancilla 2)
4:   Toffoli(control 4, ancilla 2, ancilla 3)
5:   Toffoli(control 5, ancilla 3, ancilla 4)
6:   CX(ancilla 4, target)
7:   Toffoli(control 5, ancilla 3, ancilla 4)
8:   Toffoli(control 4, ancilla 2, ancilla 3)
9:   Toffoli(control 3, ancilla 1, ancilla 2)
10:  Toffoli(control 1, control 2, ancilla 1)
11: end function
  
```

control value from one to zero, the X gate is applied before and after the gate on the control qubit. The subcircuit is outlined in pseudo-code in Algorithm 5.

For the propagation subcircuit, further division into two modules (subcircuits) is possible based on the streaming direction, left and right. As an example, we explain the construction of the right shift operator, shown in the case of five working qubits in Figure 9. This right shift subcircuit consists of three, four, and five qubit-controlled X gates. The Intel Quantum SDK enables us to create different custom gates for distinct numbers of control qubits in the general case. Here, various custom gates with MCX5, MCX4, and MCX3 are constructed following the steps provided in Section 4.1.3. These custom gate quantum kernels are invoked in the subcircuit of the right shift operator. The pseudocode in Algorithm 6 demonstrates this particular implementation.

Constructing the subcircuit and custom gates help us divide the complex algorithm into smaller modules. It helps in writing generalized and expandable programs on a higher level, giving an optimized way for writing quantum circuits. Coupling these subcircuits together as illustrated in Algorithm 2, we complete the whole algorithm for the QLBM example.

4.1.4 Post-processing. Post-processing module consists of extracting the required probabilities from the full-state simulator and deriving the macroscopic concentration values. The process to derive the concentration from the probabilities is given in Budinski [2021, Section 3.1], in particular the renormalization of the state amplitudes. These values are then fed back to the algorithm as the initial state for the next time step, until the desired time evolution is completed.

ALGORITHM 5: Subcircuit: Collision**Require:** const int (control qubit1, control qubit2, target qubit, phase angles 1–2)**Ensure:** collision subcircuit

```

1: function (QUANTUM KERNEL) COLLISION(control 1, control 2, target qubit, angles 1–2)
2:   H(qubit control 1)
3:   X(qubit control 1)
4:   X(qubit control 2)      ▷ Changing the control value of qubit from 1 to 0 using X gate
5:
6:   CCPHASE(control 1, control 2, target qubit, angle 1)
7:   Toffoli(control 1, control 2, target qubit)
8:   CCPHASE(control 1, control 2, target qubit, angle 1)
9:   Toffoli(control 1, control 2, target qubit)
10:
11:  X(qubit control 1)
12:
13:  CCPHASE(control 1, control 2, target qubit, angle 2)
14:  Toffoli(control 1, control 2, target qubit)
15:  CCPHASE(control 1, control 2, target qubit, angle 2)
16:  Toffoli(control 1, control 2, target qubit)
17:
18:  X(qubit control 1)
19:  X(qubit control 2)
20:
21:  CCPHASE(control 1, control 2, target qubit, -angle 1)
22:  Toffoli(control 1, control 2, target qubit)
23:  CCPHASE(control 1, control 2, target qubit, -angle 1)
24:  Toffoli(control 1, control 2, target qubit)
25:
26:  X(qubit control 1)
27:
28:  CCPHASE(control 1, control 2, target qubit, -angle 2)
29:  Toffoli(control 1, control 2, target qubit)
30:  CCPHASE(control 1, control 2, target qubit, -angle 2)
31:  Toffoli(control 1, control 2, target qubit)
32:
33:  H(qubit control 1)
34: end function

```

4.2 Expanding to 2D Advection-diffusion and Navier–Stokes Equations

The implementation of the 1D quantum algorithm involves formulating custom gates that can be generalized, resulting in a set of generic multi-qubit gates. This approach allows for the reuse of these custom gates in other algorithms, and new custom gates can similarly be formulated to suit other quantum circuits. In this spirit, we briefly outline an expansion to the 2D advection-diffusion [Budinski 2021] and Navier–Stokes [Budinski 2022] algorithms. These algorithms share several similarities with the 1D advection-diffusion, which simplifies their construction.

For the 2D advection-diffusion, which follows a D2Q5 lattice arrangement, the difference is mainly in the collision subcircuit with five multi-qubit controlled operators, one for each velocity.

ALGORITHM 6: Subcircuit: five-qubit right shift**Require:** const int (control qubits, target qubits)**Ensure:** right shift subcircuit

```

1: function (QUANTUM KERNEL) RIGHT_SHIFT(control qubits, target qubits)
2:   X(qubit control)
3:   MCX5(control qubits, target)
4:   MCX4(control qubits, target)
5:   MCX3(control qubits, target)
6:   CX(control, target)
7:   CX(control, target)
8:   Toffoli(control qubits, target)
9:   X(qubit control)
10: end function

```

The process of creating these operators is similar to 1D: formulate custom gates (to be specific, a three-qubit controlled phase gate) and utilize them to derive the subcircuit for the 2D collision. Similarly, for the propagation step, the circuit is divided into four smaller modules according to the directions of the necessary four shifts. Combining them together similarly to the 1D case forms the propagation subcircuit. Many custom gates from the 1D implementation can be reused but some new gates are also needed.

The implementation of the Navier–Stokes QLBM algorithm involves solving two lattice Boltzmann equations simultaneously as explained in Budinski [2022]. The collision subcircuit is completely different from the one used in 2D advection-diffusion equation, while the propagation step is carried out twice respectively for the two governing equations. The primary challenge lies in the collision subcircuit, which requires classical calculation of the diagonal elements of the non-unitary matrix and further encoding the matrix using the linear combination of operators approach. The efficient hybrid quantum-classical design is particularly valuable here. The remaining parts of the algorithm can be implemented using the modular approach (custom gates and subcircuits) in the vein of the previous discussion.

4.3 Numerical Validation of the Circuits

In this section, we implemented the quantum algorithm for 1D advection-diffusion in the Intel Quantum SDK while demonstrating the use of classical programming principles in quantum programming. Furthermore, for a simple numerical validation of the soundness of the implementation as a physics model, we compare the results after each step with the known analytical solution and the classical LBM.

The example problem considered is a transport in a channel having a fluid concentration $C(x, t)$. A Gauss-like distribution sets the initial concentration $C(x, 0)$ for the problem. The setup is the following:

- (1) A periodic boundary condition.
- (2) Number of lattice sites is a power of 2, the size is chosen so that the distributions do not overlap. We use a minimum of $2^4 = 16$ lattice sites.
- (3) For the illustration here, the number of lattice sites is 32 and the initial concentration is given as $C(5, 0) = C(7, 0) = 0.5$ and $C(6, 0) = 1.0$.

In Figure 10, we visualize the concentrations extracted after time steps 2 and 40. The initial concentration is a triangle-shape with maximum concentration at a lattice site $x = 6$ (denoted with

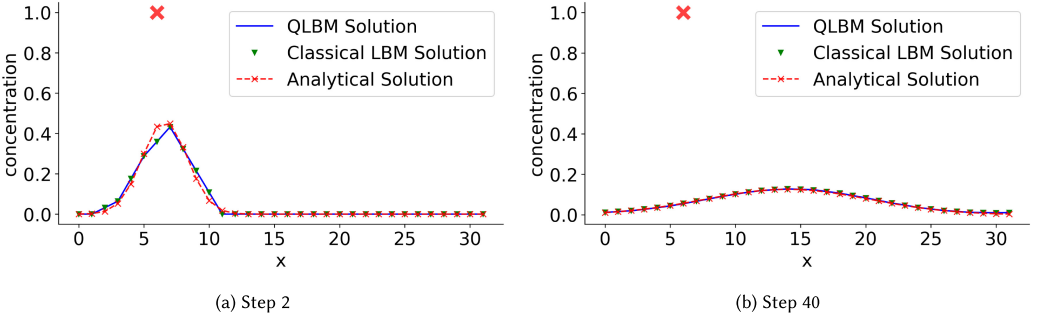


Fig. 10. Comparison between the analytical (denoted by a dotted red line with cross mark), QLBM (denoted by a blue line), and the classical lattice Boltzmann method (denoted by green triangle) results for time steps 2 and 40. The standalone red cross at the top denotes the concentration $C(6, 0) = 1.0$ as the initial condition.

a red cross), gradually decreasing towards the edge of the domain. The results show some difference with the analytical solution in the first few time steps. This is not produced by the quantum algorithm itself but is a characteristic of the Boltzmann discretization and the low resolution of the computational grid. To some extent the accuracy of the model could be increased with a finer grid providing a higher resolution and, hence, requiring a greater number of qubits for the circuit implementation. We compared these results with the classical LBM and they agree within a numerical tolerance of 10^{-12} which is ignorable, which confirms that the discrepancies are due to the inherent limitations of the discretization and grid resolution.

5 Discussion: Using Classical Techniques on Quantum SDKs

The Intel Quantum SDK differs from other quantum software development kits like Qiskit [Qiskit contributors 2023], PennyLane [Bergholm et al. 2022], and the *Munich Quantum Toolkit (MQT)* [Quetschlich et al. 2023] in several key aspects, including its target hardware, programming approach, and integration with classical computing resources. In essence, the Intel Quantum SDK is distinct in its integration with the Intel classical computing resources and its focus on high-performance and hybrid quantum-classical computing, leveraging C++ for development. In contrast, Qiskit, PennyLane, and MQT are more flexible in terms of hardware support and focus on accessibility and integration with programming languages like Python, each with its own specialized use cases and target communities. The classical programming principles outlined can be applied to these quantum SDKs with some adjustments to fit the specific features and capabilities. Here is a brief overview of the aforementioned SDKs from this perspective:

- (1) **Modularization:** All Qiskit, PennyLane, and MQT support modular programming, allowing developers to create reusable components. This principle can be applied by structuring quantum programs into functions or classes that can be easily reused and maintained.
- (2) **Hybrid design:** These SDKs also support hybrid quantum-classical computing, enabling the integration of quantum algorithms with classical pre- and post-processing. Developers can design algorithms that leverage the strengths of both quantum and classical computing.
- (3) **Custom gates:** While the specific method of creating custom gates may vary between SDKs, the concept remains the same. Developers can define new gates as combinations of existing gates to simplify complex circuit designs.

- (4) Iterative development: The iterative approach to developing and refining quantum algorithms is universally applicable. Developers can iteratively test and optimize their quantum circuits in any SDK to achieve the desired outcomes.

Overall, the principles of classical programming provide a solid foundation for developing quantum algorithms in any quantum SDK, promoting code reusability, maintainability, and efficiency. The key is to adapt these principles to the unique features and constraints of the chosen SDK. In this work, the Intel Quantum SDK has been utilized due to its emphasis on adapting such features to the C++ programming language, particularly crafted to support quantum operations to bridge the gap between traditional programming and the unique requirements of quantum computing [Wu et al. 2023]. In our experience, this integration of classical programming paradigms with quantum functionalities creates a user-friendly and efficient hybrid quantum-classical programming experience, making it accessible to a broad range of users. However, as mentioned above, similar efforts are commonplace in the quantum computing community, and we do not aim to (nor can we) place these efforts in any particular order of preference based on this work.

6 Summary

In this article, we recreated the quantum algorithm for solving the advection-diffusion equation using the LBM while explaining the principles used to generalize and expand the quantum circuits with the Intel Quantum SDK. It was shown how classical programming techniques, such as iterative development cycles and modular design methods, can be leveraged in quantum programming. The top-down modular design approach helps break down complex quantum algorithms into sub-circuits, while the creation of custom gates aids in implementing multi-qubit controlled gates. The creation of generalized circuits in terms of number of qubits and the problem variables benefits from this approach. In particular, these techniques enable one to maintain and reuse quantum kernels in the Intel Quantum SDK. This is an example of how the utilization of classical software development methodologies helps to achieve robust and beneficial quantum software code.

Appendix

A Linear Combination of Unitary Matrix Approach

The **linear combination of unitaries (LCU)** approach is a technique used in quantum computing to efficiently simulate quantum circuits and algorithms. This approach involves representing a quantum operation as a linear combination of elementary unitary operations. Unitary operations are fundamental to quantum mechanics and represent reversible transformations on quantum states.

Mathematically, this can be expressed as follows:

Suppose we have a quantum operation U that we want to represent as a linear combination of unitaries $U_1, U_2, \dots, U_n$. Then, we can write:

$$U = \sum_{i=1}^n c_i U_i,$$

where c_i are complex coefficients representing the proportions of each unitary operation U_i in the combination.

Here is a more detailed breakdown:

- U : The target quantum operation we want to represent.
- $U_1, U_2, \dots, U_n$: Elementary unitary operations or a set of predefined unitaries.
- $c_1, c_2, \dots, c_n$: Complex coefficients determining the contribution of each unitary operation in the linear combination.

In practice, determining the coefficients c_i can involve various techniques such as optimization algorithms, theoretical analysis, or experimental measurements.

LCU is particularly useful in quantum computing for efficiently simulating quantum circuits and algorithms, as it allows for the decomposition of complex quantum operations into simpler unitary operations, making simulations more tractable and scalable.

Let's consider the example of a single-qubit quantum operation, the Hadamard gate (H gate). The Hadamard gate is a fundamental gate in quantum computing that creates superposition. It can be represented by the following matrix:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Now, let's decompose the Hadamard gate into a linear combination of elementary unitary operations. We can represent the Hadamard gate as a linear combination of the Pauli matrices, which are elementary unitary operations. The Pauli matrices are denoted as

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$\sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Now, we can express the Hadamard gate (H) as a linear combination of the Pauli matrices:

$$H = \frac{1}{\sqrt{2}} \sigma_x + \frac{1}{\sqrt{2}} \sigma_z.$$

This representation shows that the Hadamard gate can be decomposed into a linear combination of the Pauli matrices σ_x and σ_z .

Acknowledgment

We would especially like to thank and appreciate the entire Intel family for their key support. Among these are Prof. Dr. Anne Matsuura and Dr. Kevin Rasch who were consistently guiding and encouraging us throughout the process. Lastly, T. S. would like to appreciate all his colleagues and teachers for their academic and moral support throughout the journey.

References

- A. A. Mohamad. 2019. *Lattice Boltzmann Method—Fundamentals and Engineering Applications with Computer Codes*. Springer-Verlag London Ltd.
- Andris Ambainis. 2010. Variable time amplitude amplification and a faster quantum algorithm for solving systems of linear equations. (2010). DOI : <https://doi.org/10.48550/arXiv.1010.4458>
- Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shah Nawaz Ahmed, Vishnu Ajith, M. Sohaib Alam, Guillermo Alonso-Linaje, B. Akash Narayanan, Ali Asadi, Juan Miguel Arrazola, Utkarsh Azad, Sam Banning, Carsten Blank, Thomas R. Bromley, Benjamin A. Cordier, Jack Ceroni, Alain Delgado, Olivia Di Matteo, Amintor Dusko, Tanya Garg, Diego Guala, Anthony Hayes, Ryan Hill, Aroosa Ijaz, Theodor Isaacson, David Ittah, Soran Jahangiri, Prateek Jain, Edward Jiang, Ankit Khandelwal, Korbinian Kottmann, Robert A. Lang, Christina Lee, Thomas Loke, Angus Lowe, Keri McKiernan, Johannes Jakob Meyer, J. A. Montañez-Barrera, Romain Moyard, Zeyue Niu, Lee James O'Riordan, Steven Oud, Ashish Panigrahi, Chae-Yeun Park, Daniel Polatajko, Nicolás Quesada, Chase Roberts, Nahum Sá, Isidor Schoch, Borun Shi, Shuli Shu, Sukin Sim, Arshpreet Singh, Ingrid Strandberg, Jay Soni, Antal Száva, Slimane Thabet, Rodrigo A.

- Vargas-Hernández, Trevor Vincent, Nicola Vitucci, Maurice Weber, David Wierichs, Roeland Wiersema, Moritz Willmann, Vincent Wong, Shaoming Zhang, and Nathan Killoran. 2022. PennyLane: Automatic differentiation of hybrid quantum-classical computations. (2022). DOI : <https://doi.org/10.48550/arXiv.1811.04968>
- Ljubomir Budinski. 2021. Quantum algorithm for the advection–diffusion equation simulated with the lattice Boltzmann method. *Quantum Information Processing* 20, 2 (2021), 57. DOI : <https://doi.org/10.1007/s11128-021-02996-3>
- Ljubomir Budinski. 2022. Quantum algorithm for the Navier–Stokes equations by using the streamfunction–vorticity formulation and the lattice Boltzmann method. *International Journal of Quantum Information* 20, 02 (2022), 2150039. DOI : <https://doi.org/10.1142/s0219749921500398>
- Ljubomir Budinski, Ossi Niemimäki, Roberto Zamora-Zamora, and Valtteri Lahtinen. 2023. Efficient parallelization of quantum basis state shift. *Quantum Science and Technology* 8, 4 (2023), 045031. DOI : <https://doi.org/10.1088/2058-9565/acfab7>
- Yudong Cao, Jonathan Romero, Jonathan P. Olson, Matthias Degroote, Peter D. Johnson, Má ria Kieferová, Ian D. Kivlichan, Tim Menke, Borja Peropadre, Nicolas P. D. Sawaya, Sukin Sim, Libor Veis, and Alán Aspuru-Guzik. 2019. Quantum Chemistry in the age of quantum computing. *Chemical Reviews* 119, 19 (2019), 10856–10915. DOI : <https://doi.org/10.1021/acs.chemrev.8b00803>
- Andrew M. Childs. 2009. Universal computation by quantum walk. *Physical Review Letters* 102, 18 (2009), 180501. DOI : <https://doi.org/10.1103/PhysRevLett.102.180501>
- Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. 2017. Open quantum assembly language. (2017). DOI : <https://doi.org/10.48550/arXiv.1707.03429>
- Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. (2014). DOI : <https://doi.org/10.48550/arXiv.1411.4028>
- Siddhant Garg and Goutham Ramakrishnan. 2020. Advances in quantum deep learning: An overview. (2020). DOI : <https://doi.org/10.48550/arXiv.2005.04316>
- Iordanis Kerenidis and Anupam Prakash. 2020. Quantum gradient descent for linear systems and least squares. *Physical Review A* 101, 2 (2020), 022316. DOI : <https://doi.org/10.1103/physreva.101.022316>
- Pradnya Khalate, Xin-Chuan Wu, Shavindra Premaratne, Justin Hogaboam, Adam Holmes, Albert Schmitz, Gian Giacomo Guerreschi, Xiang Zou, and A. Y. Matsuura. 2022. An LLVM-based C++ Compiler toolchain for variational hybrid quantum-classical algorithms and quantum accelerators. (2022). DOI : <https://doi.org/10.48550/arXiv.2202.11142>
- Arif Ali Khan, Mahdi Fahmideh, Aakash Ahmad, Muhammad Waseem, Mahmood Niazi, Valtteri Lahtinen, and Tommi Mikkonen. 2022. Embracing iterations in quantum software: A vision. In *Proceedings of the 1st International Workshop on Quantum Programming for Software Engineering (QP4SE 2022)*. Association for Computing Machinery, New York, NY, USA, 11–14. DOI : <https://doi.org/10.1145/3549036.3562057>
- Timm Krueger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Viggen. 2016. *The Lattice Boltzmann Method: Principles and Practice*. Springer, Cham, Switzerland.
- Seth Lloyd, Giacomo De Palma, Can Gokler, Bobak Kiani, Zi-Wen Liu, Milad Marvian, Felix Tennie, and Tim Palmer. 2020. Quantum algorithm for nonlinear differential equations. (2020). DOI : <https://doi.org/10.48550/arXiv.2011.06571>
- Guang Hao Low and Isaac L. Chuang. 2019. Hamiltonian simulation by qubitization. *Quantum* 3 (July 2019), 163. DOI : <https://doi.org/10.22331/q-2019-07-12-163>
- Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Infomation*. Cambridge University Press, Cambridge.
- Peng Qian, Wei-Cong Huang, and Gui-Lu Long. 2019. A quantum algorithm for solving systems of nonlinear algebraic equations. (2019). DOI : <https://doi.org/10.48550/arXiv.1903.05608>
- Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. (2024). DOI : <https://doi.org/10.48550/arXiv.2405.08810>
- Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. MQT Bench: Benchmarking software and design automation tools for quantum computing. *Quantum* 7 (July 2023), 1062. DOI : <https://doi.org/10.22331/q-2023-07-20-1062>
- Merel A. Schalkers and Matthias Möller. 2024. Momentum exchange method for quantum Boltzmann methods. (2024). DOI : <https://doi.org/10.48550/arXiv.2404.17618>
- Merel A. Schalkers and Matthias Möller. 2024. Efficient and fail-safe quantum algorithm for the transport equation. *Journal of Computational Physics* 502 (2024), 112816. DOI : <https://doi.org/10.1016/j.jcp.2024.112816>
- Siddharth Sharma. 2020. QEML (Quantum Enhanced Machine Learning): Using Quantum Computing to Enhance ML Classifiers and Feature Spaces. (2020). DOI : <https://doi.org/10.48550/arXiv.2002.10453>
- V. V. Shende, S. S. Bullock, and I. L. Markov. 2006. Synthesis of quantum-logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 6 (2006), 1000–1010. DOI : <https://doi.org/10.1109/tcad.2005.855930>

- M. Mitchell Waldrop. 2016. The chips are down for Moore's law. *Nature* 530, 7589 (2016), 144–147. DOI : <https://doi.org/10.1038/530144a>
- Xin-Chuan Wu, Shavindra Premaratne, and Kevin Rasch. 2023. Invited paper: Introduction to hybrid quantum-classical programming using C++ quantum extension. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD'23)*. 1–6. DOI : <https://doi.org/10.1109/ICCAD57390.2023.10323946>
- Xiao-Ming Zhang, Tongyang Li, and Xiao Yuan. 2022. Quantum state preparation with optimal circuit depth: Implementations and applications. *Physical Review Letters* 129, 23 (2022), 230504. DOI : <https://doi.org/10.1103/physrevlett.129.230504>
- Jianjun Zhao. 2020. Quantum Software Engineering: Landscapes and Horizons. (2020). DOI : <https://doi.org/10.48550/arXiv.2007.07047>
- Julien Zylberman and Fabrice Debbsch. 2024. Efficient quantum state preparation with Walsh series. *Phys. Rev. A* 109, 4 (April 2024), 042401. DOI : <https://doi.org/10.1103/PhysRevA.109.042401>

Received 29 September 2023; revised 4 June 2024; accepted 21 June 2024